Fatec
Registro



TÉCNICAS DE PROGRAMAÇÃO II

PROFº LUIZ CLÁUDIO

MONGO DB – BANCO DE DADOS NÃO RELACIONAL - NOSQL

Utilizando Mongo DB





MONGO DB – INSTALAÇÃO MONGO

1º Passo Instalar a biblioteca pymongo digitar o comando no cmd do Windows.

Comando -> `pip install pymongo`

2º Digitação do Código

```
from tkinter import *
from tkinter import ttk
import tkinter as tk
import pymongo
```

Biblioteca para utilização do combobox

Biblioteca do mongodb

MONGO DB – CONFIGURAÇÕES

Configuração da tela

```
class CadastroClientes:

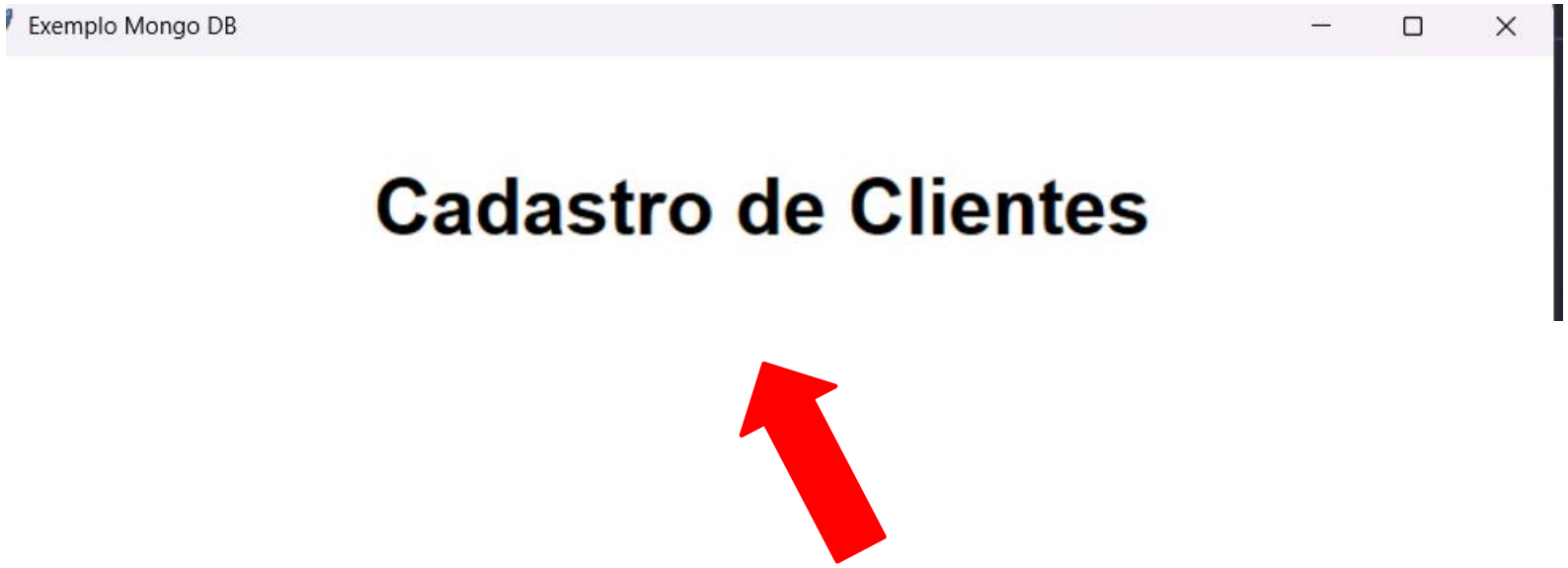
    def __init__(self):
        self.tela = Tk()
        self.tela.title("Exemplo Mongo DB")
        self.tela.geometry("800x600")
        self.tela.configure(background="#ffffff")
```

Configuração do mongodb

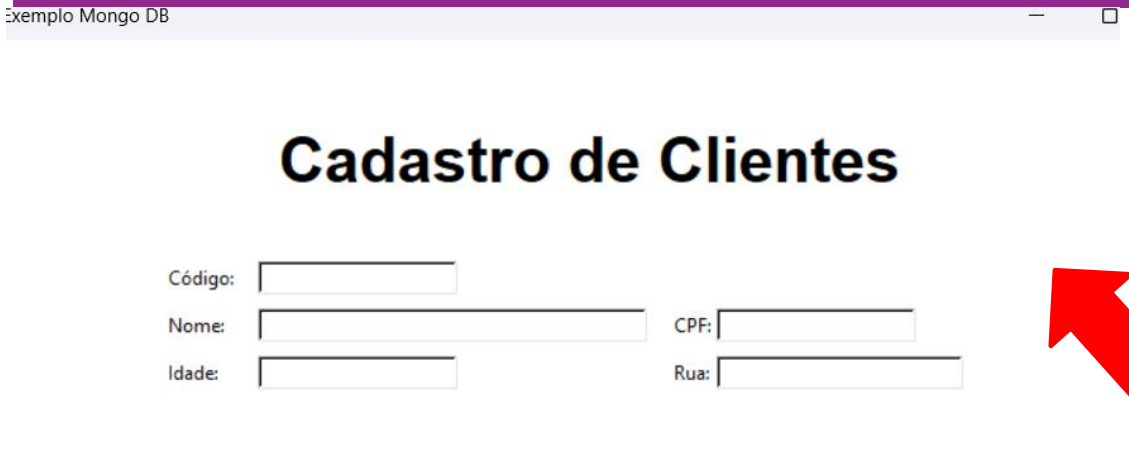
```
        self.conexao = pymongo.MongoClient("mongodb://localhost:27017/")
        self.db = self.conexao["exemplo_clientes"]
        self.collection = self.db["clientes"]
```

Nome da coleção

MONGO DB – BANCO DE DADOS NÃO RELACIONAL



MONGO DB – CONSTRUÇÃO FORMULÁRIO




```
def criar_componentes(self):

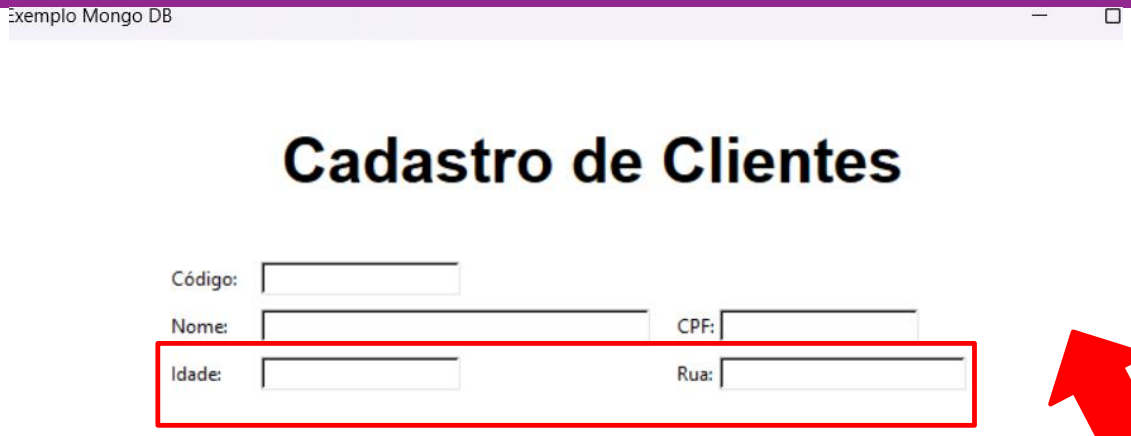
    Label(self.tela, text="Cadastro de Clientes",font=("Arial", 30, "bold"),bg="#ffffff").place(x=200, y=50)

    Label(self.tela, text="Código:", bg="#ffffff").place(x=130, y=140)
    self.txt_codigo = Entry(self.tela, width=20)
    self.txt_codigo.place(x=190, y=140)

    Label(self.tela, text="Nome:", bg="#ffffff").place(x=130, y=170)
    self.txt_nome = Entry(self.tela, width=40)
    self.txt_nome.place(x=190, y=170)

    Label(self.tela, text="CPF:", bg="#ffffff").place(x=450, y=170)
    self.txt_cpf = Entry(self.tela, width=20)
    self.txt_cpf.place(x=480, y=170)
```

MONGO DB – CONSTRUÇÃO FORMULÁRIO




```
Label(self.tela, text="Idade:", bg="#ffffff").place(x=130, y=200)
self.txt_idade = Entry(self.tela, width=20)
self.txt_idade.place(x=190, y=200)

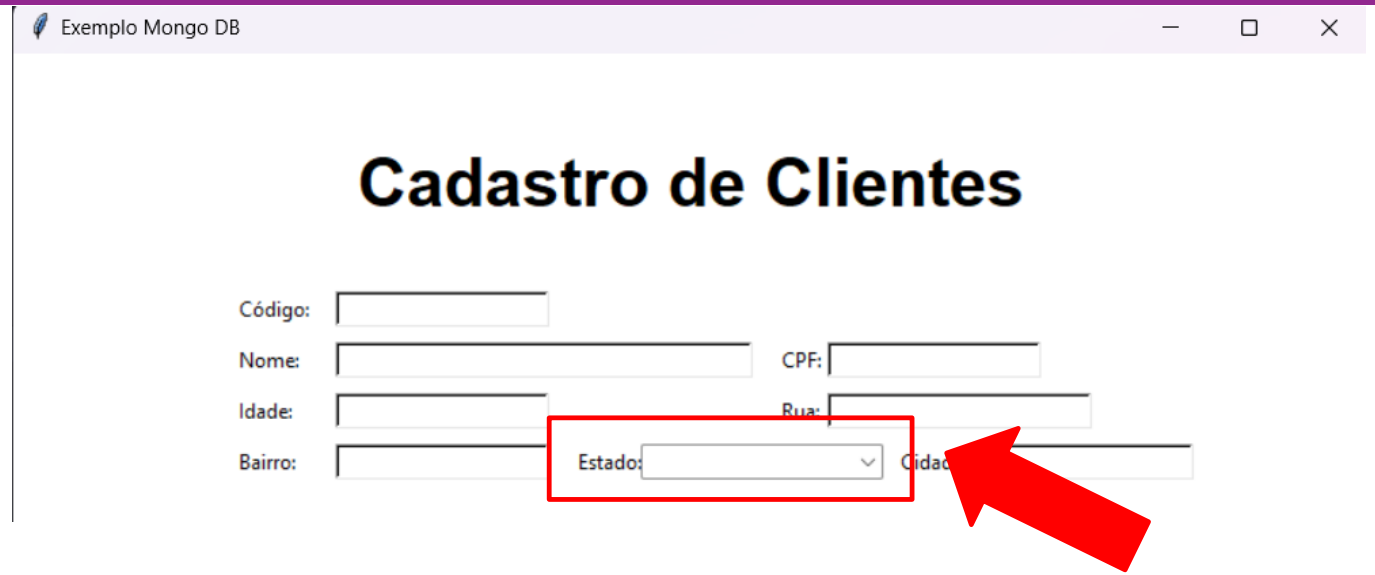
Label(self.tela, text="Rua:", bg="#ffffff").place(x=450, y=200)
self.txt_end = Entry(self.tela, width=25)
self.txt_end.place(x=480, y=200)
```

MONGO DB – CONSTRUÇÃO FORMULÁRIO




```
Label(self.tela, text="Bairro:", bg="#ffffff").place(x=130, y=230)
self.txt_bairro = Entry(self.tela, width=20)
self.txt_bairro.place(x=190, y=230)
```

MONGO DB – CONSTRUÇÃO FORMULÁRIO




```
Label(self.tela, text="Estado:", bg="#ffffff").place(x=330, y=230)
self.comboestado = ttk.Combobox(self.tela, values=[
    "São Paulo", "Rio de Janeiro", "Minas Gerais", "Espírito Santo"
])
self.comboestado.place(x=370, y=230)
```

MONGO DB – CONSTRUÇÃO FORMULÁRIO



MONGO DB – CONSTRUÇÃO FORMULÁRIO

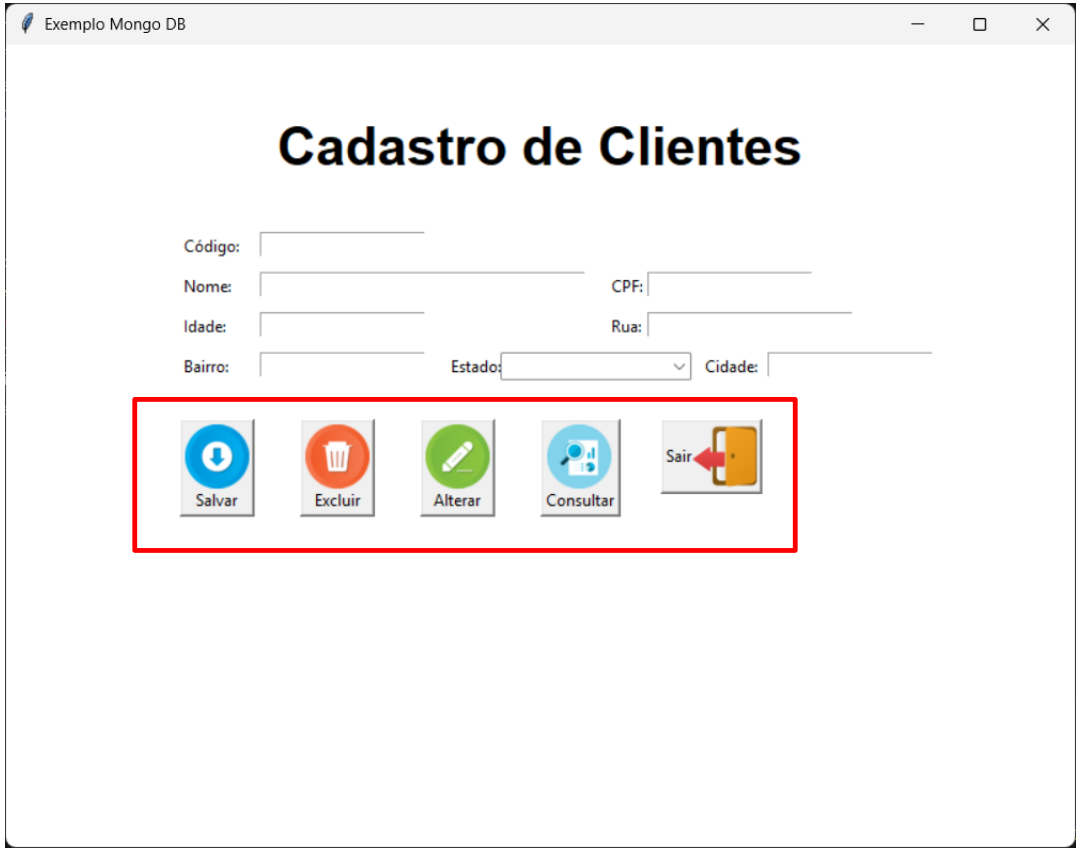



```
Label(self.tela, text="Cidade:", bg="#ffffff").place(x=520, y=230)
self.txt_cidade = Entry(self.tela, width=20)
self.txt_cidade.place(x=570, y=230)

self.lbl_resultado = Label(self.tela, text="", bg="#ffffff")
self.lbl_resultado.place(x=490, y=410)
```

Label Resultado para mostrar os dados

MONGO DB – CRIANDO OS BOTÕES




```
self.foto_salvar = PhotoImage(file="icones/salvar.png")
self.foto_excluir = PhotoImage(file="icones/excluir.png")
self.foto_alterar = PhotoImage(file="icones/alterar.png")
self.foto_consultar = PhotoImage(file="icones/consultar.png")
self.foto_sair = PhotoImage(file="icones/sair.png")


self.btn_salvar = Button(self.tela, text="Salvar",image=self.foto_salvar,compound=TOP,command=self.salvar)
self.btn_salvar.place(x=130, y=280)

self.btn_excluir = Button(self.tela, text="Excluir",image=self.foto_excluir,compound=TOP,command=self.excluir)
self.btn_excluir.place(x=220, y=280)

self.btn_alterar = Button(self.tela, text="Alterar",image=self.foto_alterar,compound=TOP,command=self.atualizar)
self.btn_alterar.place(x=310, y=280)

self.btn_consultar = Button(self.tela, text="Consultar",image=self.foto_consultar,compound=TOP,command=self.consultar)
self.btn_consultar.place(x=400, y=280)

self.btn_sair = Button(self.tela, text="Sair",image=self.foto_sair,compound=RIGHT,command=self.sair)
self.btn_sair.place(x=490, y=280)
```

Cada botão chama uma função

MONGO DB – FUNÇÃO SALVAR

```
def salvar(self):
    try:
        cliente = {
            "código": self.txt_codigo.get(),
            "nome": self.txt_nome.get(),
            "idade": int(self.txt_idade.get()),
            "endereço": self.txt_end.get(),
            "cpf": self.txt_cpf.get(),
            "bairro": self.txt_bairro.get(),
            "cidade": self.txt_cidade.get(),
            "estado": self.comboestado.get()
        }

        self.collection.insert_one(cliente)
        self.limpar()
        self.lbl_resultado.config(text="Salvo com sucesso!")

    except:
        self.lbl_resultado.config(text="Erro ao salvar")

def atualizar(self):
    codigo = self.txt_codigo.get()
```



Cada variavel ligada a uma caixa de texto

MONGO DB – INSERÇÃO DOS DADOS DICIONÁRIO DOS DADOS

```
cliente = {
    "código": self.txt_codigo.get(),
    "nome": self.txt_nome.get(),
    "idade": int(self.txt_idade.get()),
    "endereço": self.txt_end.get(),
    "cpf": self.txt_cpf.get(),
    "bairro": self.txt_bairro.get(),
    "cidade": self.txt_cidade.get(),
    "estado": self.comboestado.get()
}

self.collection.insert_one(cliente)
```



Dicionário chamado cliente está sendo criado. Este dicionário é usado para representar um conjunto de informações relacionadas a um cliente. Cada campo do cliente é representado como uma chave (por exemplo, “código", "nome", "idade", etc.) e o valor associado a cada chave é obtido de variáveis ou valores existentes no código, como selftxtcodigo, selftxtnome, self.txtidade, etc.

collection é uma instância de um objeto que representa uma coleção de um banco de dados NoSQL (como o MongoDB). A função insert_one é chamada para inserir o dicionário cliente na coleção.

MONGO DB – LIMPANDO CAIXAS DE TEXTO DEPOIS DE SALVAR

```
def limpar(self):
    self.txt_codigo.delete(0, END)
    self.txt_nome.delete(0, END)
    self.txt_idade.delete(0, END)
    self.txt_end.delete(0, END)
    self.txt_bairro.delete(0, END)
    self.txt_cidade.delete(0, END)
    self.txt_cpf.delete(0, END)
    self.comboestado.set("")
```



Após a inserção dos dados as caixas de textos, será apagada automaticamente

MONGO DB – ATUALIZAÇÃO DOS DADOS - UPDATE

```python
def atualizar(self):
    codigo = self.txt_codigo.get()

    self.collection.update_one(
        {"código": codigo},
        {"$set": {
            "nome": self.txt_nome.get(),
            "idade": int(self.txt_idade.get()),
            "endereço": self.txt_end.get(),
            "cpf": self.txt_cpf.get(),
            "bairro": self.txt_bairro.get(),
            "cidade": self.txt_cidade.get(),
            "estado": self.comboestado.get()
        }}
    )

    self.limpar()
    self.lbl_resultado.config(text="Atualizado!")
```



O método update_one é comumente usado em bancos de dados NoSQL, como o MongoDB, para atualizar um único documento que atende a um critério específico.

MONGO DB – ATUALIZAÇÃO DOS DADOS - UPDATE



```
self.collection.update_one(
    {"código": codigo},
    {"$set": {
        "nome": self.txt_nome.get(),
        "idade": int(self.txt_idade.get()),
        "endereço": self.txt_end.get(),
        "cpf": self.txt_cpf.get(),
        "bairro": self.txt_bairro.get(),
        "cidade": self.txt_cidade.get(),
        "estado": self.comboestado.get()
```

O segundo argumento é outro dicionário que usa a operação $set para especificar quais campos e valores devem ser atualizados no registro correspondente, todos os campos estão sendo atualizados com os valores das variáveis correspondentes, como "código", "nome", "idade", "endereço", "cpf", "bairro", "cidade" e "estado".

MONGO DB – APAGAR OS DADOS - DELETE

```python
def excluir(self):
    codigo = self.txt_codigo.get()
    self.collection.delete_one({"código": codigo})
    self.limpar()
    self.lbl_resultado.config(text="Excluído!")
```

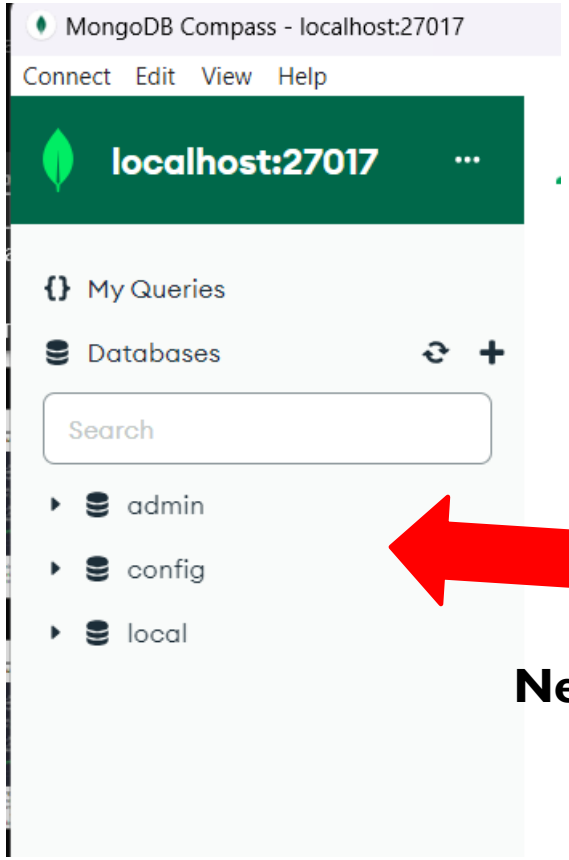


Função utiliza a função delete_one para excluir um registro no banco de dados.
A exclusão é feita com base na correspondência do campo "código" do registro com o valor armazenado na variável codigo.

O único argumento para a função delete_one é um dicionário que especifica o critério de pesquisa. Neste caso, o critério é que o campo "código" no banco de dados corresponda ao valor da variável codigo.

MONGO DB – REALIZANDO O TESTE DA APLICAÇÃO - SALVAR

Realizando o teste da aplicação - Salvar



Nenhuma base existente

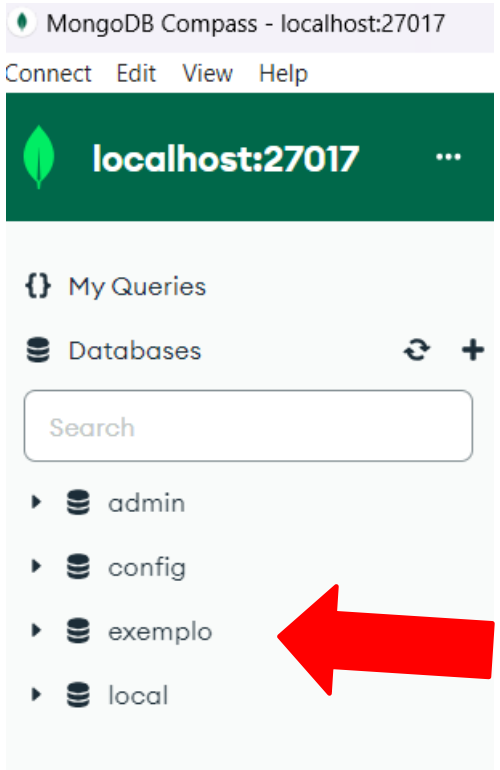


Preencha os campos

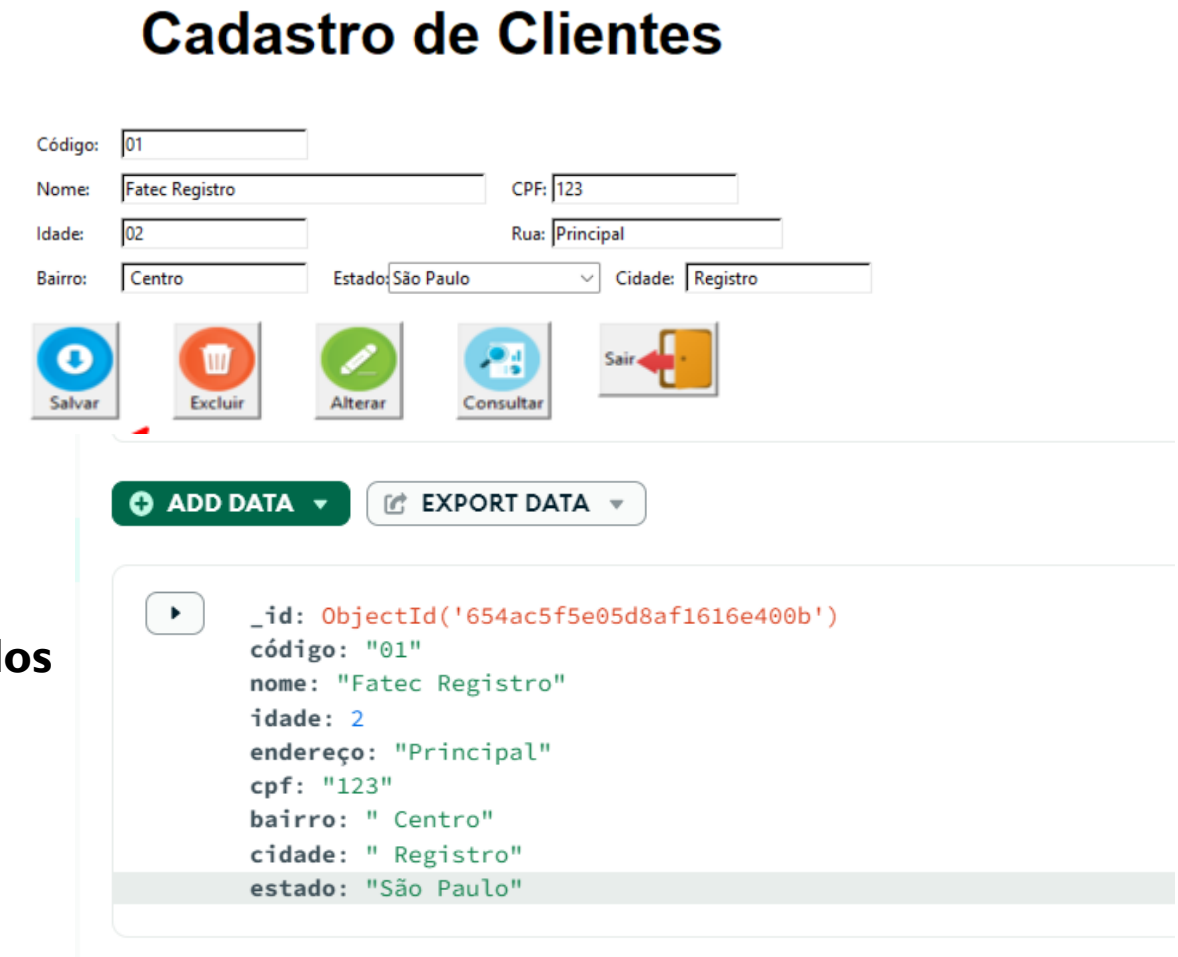
Click no botão Salvar

MONGO DB – REALIZANDO O TESTE DA APLICAÇÃO - SALVAR

Atualize o mongo db

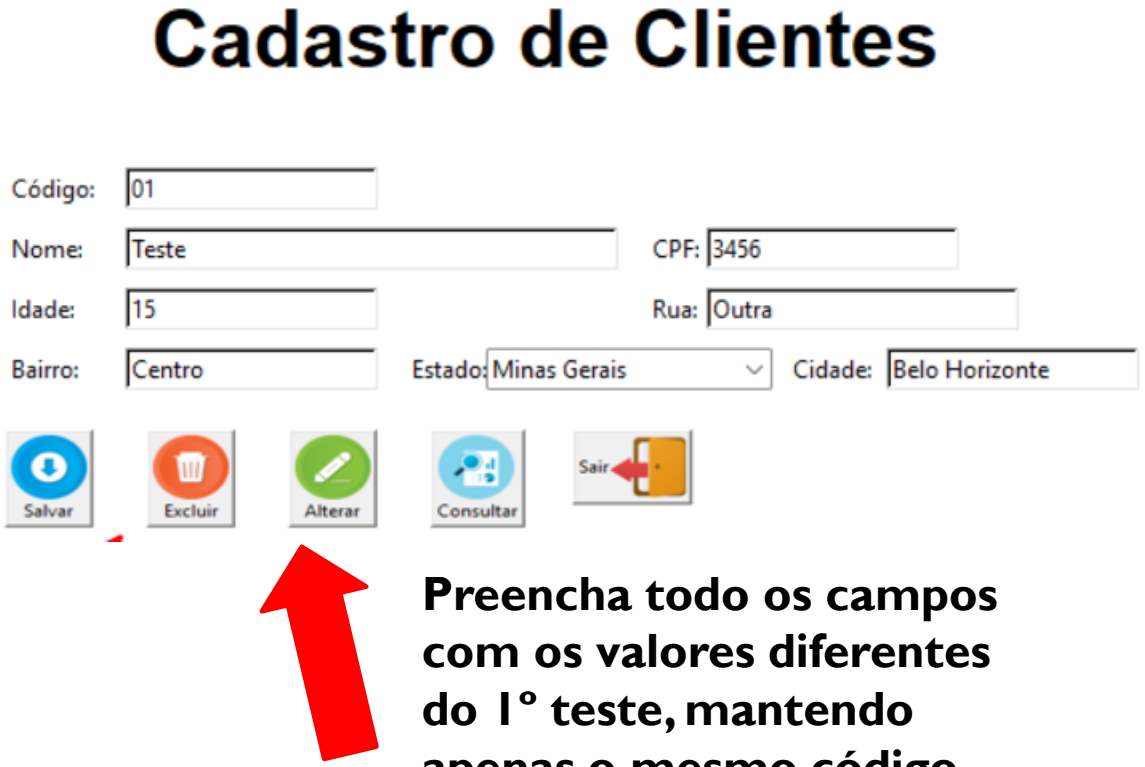


Base de dados



MONGO DB – REALIZANDO O TESTE DA APLICAÇÃO - ALTERAR

Teste Alterar



Preencha todo os campos com os valores diferentes do 1° teste, mantendo apenas o mesmo código (01).

```
_id: ObjectId('654ac5f5e05d8af1616e400b')
código: "01"
nome: "Teste"
idade: 15
endereço: "Outra"
cpf: "3456"
bairro: "Centro"
cidade: "Belo Horizonte"
estado: "Minas Gerais"
```

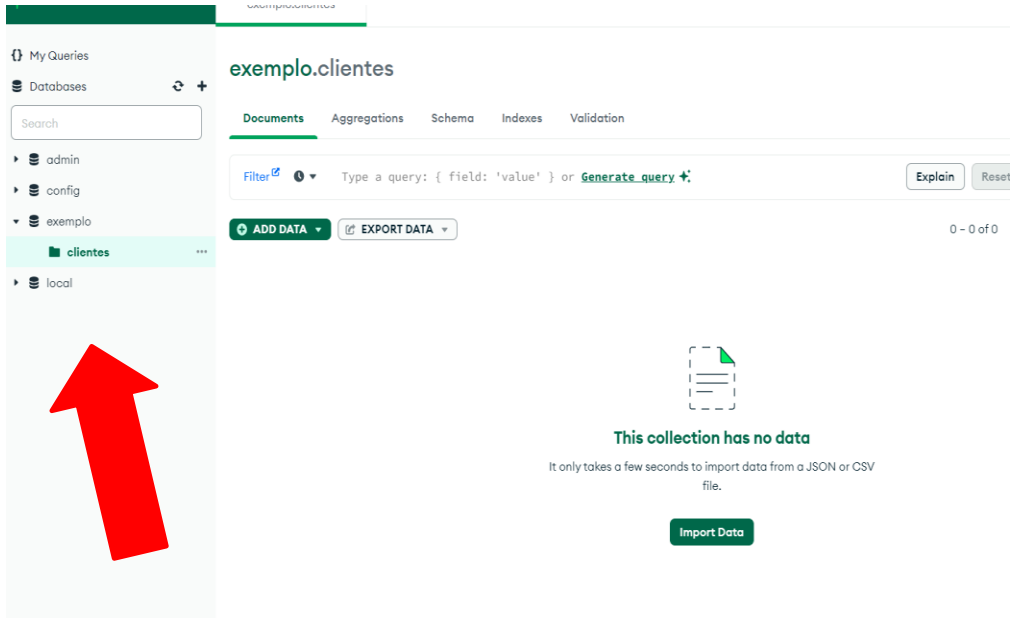
Atualizado no Mongo DB

MONGO DB – REALIZANDO O TESTE DA APLICAÇÃO - EXCLUIR

Teste Excluir



Digite no campo código – 01
Click em Excluir



Nenhuma Collection Existente

MONGO DB – CONSULTAR OS DADOS

No código existente adicionar a função

```
def consultar(self):

    codigo = self.txt_codigo.get()

    resultado = self.collection.find_one({"código": codigo})

    if resultado:
        self.txt_nome.insert(END, resultado["nome"])
        self.txt_idade.insert(END, resultado["idade"])
        self.txt_end.insert(END, resultado["endereço"])
        self.txt_cpf.insert(END, resultado["cpf"])
        self.txt_bairro.insert(END, resultado["bairro"])
        self.txt_cidade.insert(END, resultado["cidade"])
        self.comboestado.set(resultado["estado"])
    else:
        self.lbl_resultado.config(text="Não encontrado")
```

MONGO DB – CONSULTAR OS DADOS

Explicação do Código

```
def consultar(self):

    codigo = self.txt_codigo.get()
```

codigo = self.txt_codigo.get(): Esta linha obtém o valor digitado no campo de entrada self.txt_codigo e armazena-o na variável codigo.

O código é usado como critério de pesquisa para localizar um cliente no banco de dados.

```
def consultar(self):

    codigo = self.txt_codigo.get()

    resultado = self.collection.find_one({"código": codigo})
```

resultado = collection.find_one({"código": codigo}): Aqui, a função find_one é usada para buscar um documento no banco de dados MongoDB na coleção collection. O critério de pesquisa é o campo "código" no banco de dados, que é comparado ao valor armazenado na variável codigo. A função find_one retorna o primeiro documento que atende ao critério de pesquisa ou None se nenhum documento for encontrado.

MONGO DB – CONSULTAR OS DADOS

```
def consultar(self):

    codigo = self.txt_codigo.get()

    resultado = self.collection.find_one({"código": codigo})

    if resultado:
        self.txt_nome.insert(END, resultado["nome"])
        self.txt_idade.insert(END, resultado["idade"])
        self.txt_end.insert(END, resultado["endereço"])
        self.txt_cpf.insert(END, resultado["cpf"])
        self.txt_bairro.insert(END, resultado["bairro"])
        self.txt_cidade.insert(END, resultado["cidade"])
        self.comboestado.set(resultado["estado"])
    else:
        self.lbl_resultado.config(text="Não encontrado")
```

Se um resultado for encontrado (if resultado), os detalhes do cliente são exibidos nos campos de entrada txt_nome, txt_idade, txt_end, txt_cpf, txt_bairro, txt_cidade, e no widget comboestado.

Caso nenhum resultado seja encontrado, a label lbl_resultado é atualizada com o texto "Nenhum resultado encontrado".
.

MONGO DB – REALIZANDO O TESTE DA APLICAÇÃO - CONSULTAR

Teste Consultar



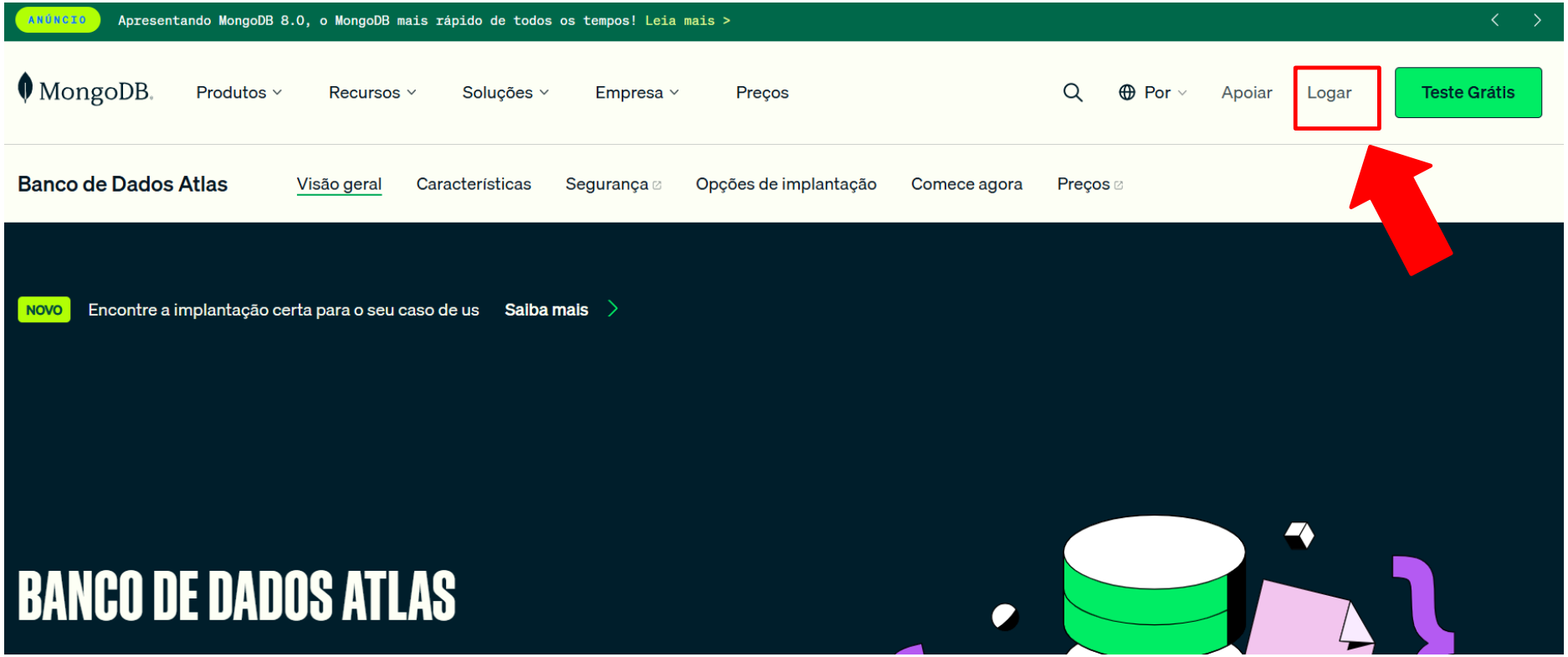
Digite o campo código o valor existente no mongo db e depois click em consultar



Resultado

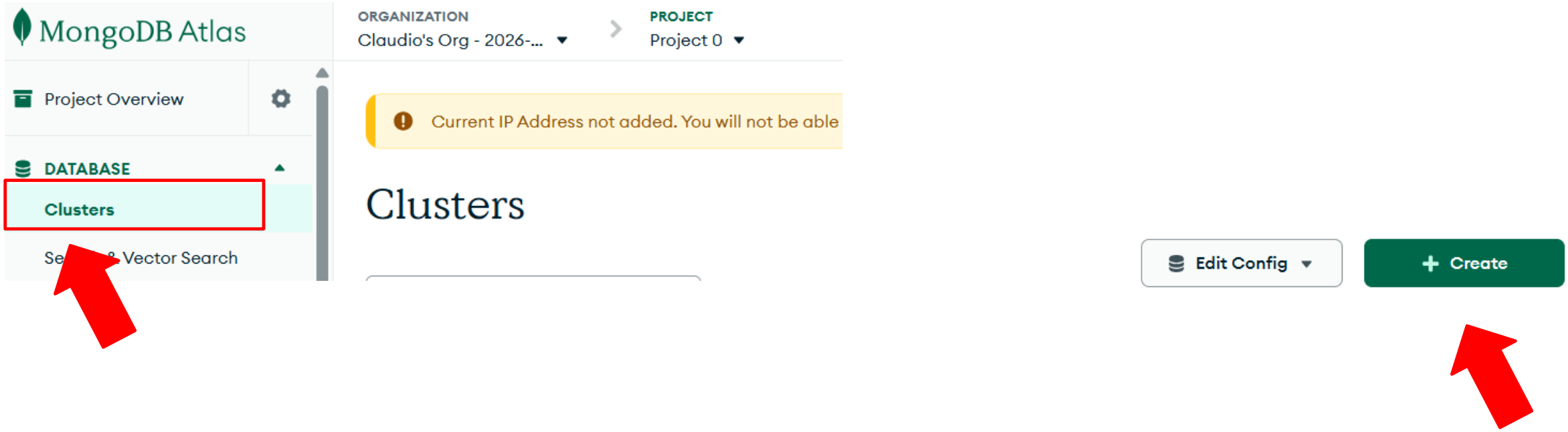
MONGO DB – UTILIZANDO MONGO DB ATLAS ON LINE

Entre no site https://www.mongodb.com/ , faça um cadastro



MONGO DB – UTILIZANDO MONGO DB ATLAS ON LINE

Clique em Cluster e depois create...



MONGO DB ATLAS – CONFIGURAÇÕES

Configuração mongo com Atlas

```
# CONEXÃO COM O MONGODB ATLAS
self.conexao = pymongo.MongoClient(
    "mongodb+srv://claudiobarreto83_db_user:einKI8Oshv5d5ZPP@teste.n0ftves.mongodb.net"
)

# CRIA O BANCO DE DADOS
self.db = self.conexao["exemplo_clientes"]

# CRIA COLEÇÃO
self.collection = self.db["clientes"]
```

Coloque aqui a Uri do cluster do atlas

Nome coleção

Nome do banco que será criado



LUIZ.BARRETO@CPS.SP.GOV.BR